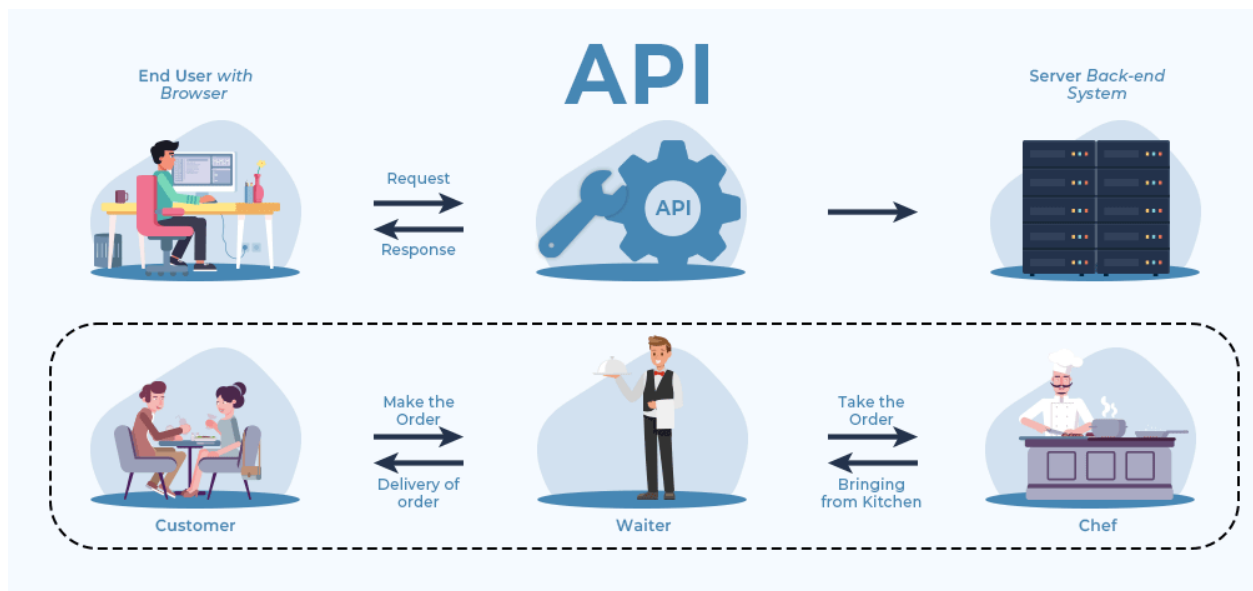


APIs

Un API (Application Programming Interface) este un set de reguli care permite comunicarea între aplicații
Intermediar între cererea unui utilizator și răspunsul serverului,
asemănător unui chelner într-un restaurant



1. Clientul trimite o cerere prin API către server
 - 1.5. API-ul o trimite către server
2. Serverul răspunde
 - 2.5. API-ul returnează datele clientului

Structura unei cereri HTTP

URL - Adresa specifică a resursei dorite

Headers - Informații adiacente cererii (ex.: autentificare)

Body - Date trimise în corpul cererii (ex.: pentru POST și PUT)

Exemplu de Cerere GET:

GET /contact HTTP/1.1

Host: example.com

User-Agent: curl/8.6.0

Accept: */*

Coduri de Răspuns:

- 200 OK - Cererea a fost procesată cu succes
- 404 Not Found - Resursa cerută nu a fost găsită
- 500 Server Error - Eroare la nivel de server

REST API

REST (REpresentational State Transfer) este un stil arhitectural pentru proiectarea API-urilor. REST API-urile sunt foarte populare deoarece sunt ușor de înțeles și utilizat, iar interacțiunile sunt făcute pe baza protocoalelor HTTP.

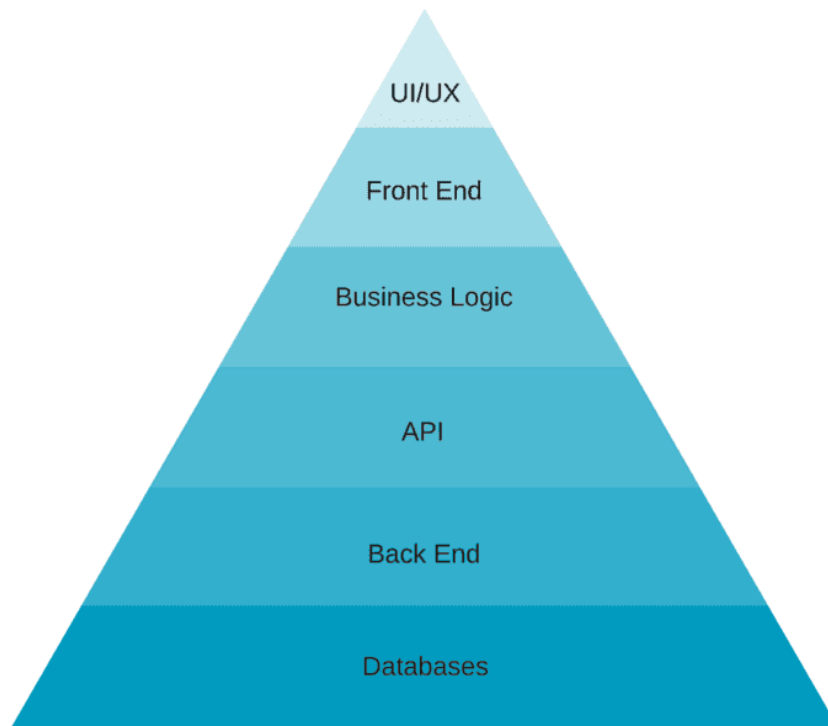
Principii REST:

- **Stateless** - Fiecare cerere către server este independentă
- **Resurse Identificabile** - Fiecare resursă (ex.: utilizatori, produse) are un URL unic
- **Operații Standardizate** - Metode HTTP standard (GET, POST, etc.) sunt folosite pentru operații asupra resurselor

În REST API, metodele HTTP definesc tipul acțiunii care va fi efectuată asupra resursei cerute.

- **GET** - Obține date.
GET /api/v1/users/1 - Obține datele utilizatorului cu ID-ul 1
- **POST** - Creează o nouă resursă.
POST /api/v1/users - Creează un nou utilizator cu date specificate
- **PUT** - Actualizează o resursă.
PUT /api/v1/users/1 - Actualizează datele utilizatorului cu ID-ul 1
- **DELETE** - Șterge o resursă.
DELETE /api/v1/users/1 - Șterge utilizatorul cu ID-ul 1

Full Stack Developer Skillset



Client Storage

Client storage = Informație stocată pe calculatorul unui client, mult după ce a închis website-ul.

Client storage include cookies, local storage, cache.

Cookies = un dicționar de tip String-Obiect salvat local.

Specificații de bază:

- Mărime maximă 4,096 bytes
- Garantat 50 cookies per domain
- Garantat 3,000 cookies în total

Dacă un cookie are atributul HttpOnly, nu poate fi accesat de

JavaScript. Folosit pentru a preveni un atac de tipul **Cross-Site Scripting (XSS)** .

Dacă un cookie are atributul Secure, nu poate fi trimis decât prin HTTPS, deci doar printr-un protocol encriptat.

SameSite restricționează domeniul în care poate fi trimis un cookie, pentru a preveni **CSRF**.

None = poate fi trimis în Cross-Site requests.

Strict = poate fi trimis doar prin Same-Site requests.

Lax = poate fi trimis prin Cross-Site requests, doar dacă a fost un top-level access.

JWT

JWT - JSON Web Token -> Un obiect JSON encriptat cu base64, de obicei folosit pentru a reprezenta o sesiune activă a unui user.

LocalStorage

Salvat în localStorage sau cookies, trimis printr-un header HTTP, în funcție de valoarea sa serverul dispune sau nu resurse utilizatorului.

- Storage Persistent (nu expiră)
- Mult mai mare comparativ cu cookies (~10MB)
- Nu este trimis cu fiecare request
- Mult mai vulnerabil (poate fi citit de ORICE javascript)

Cache

Caching = În loc să cer o resursă de la X, o iau din cache, care e mai rapid

Caching-ul este folosit în multe contexte, dar în contextul web avem:

- Browser cache (eu am cache-ul)
- CDN cache (un server aproape de mine are cache-ul)
- Server-side cache (serverul are cache-ul)

Stochează resurse statice local (HTML, CSS, JS, imagini)

- Când intrăm pe un website, facem o cerere HTTP către un server
- Serverul ne dă pentru fiecare fișier de care avem nevoie și **caching headers**.
- Browser-ul stochează datele în cache.
- Când mai intrăm iar pe același website, browser-ul poate folosi versiunea din cache.

Poate funcționa ca la cookies; cu o limită de timp.

- Timp de o oră, browser-ul nu va cere acest fișier din nou.
- Nu se fac cereri extra către server

Cum știe browser-ul nostru că datele din cache sunt cele mai recente?

Primim și ținem minte:

Când vrem să folosim resursa, întrebăm server-ul:

Concluzii

- **REST API**-urile simplifică interacțiunea cu resursele web și asigură o structură clară, ușor de utilizat pentru dezvoltare și integrare.
- Client storage-ul rezolvă diferite probleme (autentificare, preferințe, eficiență) prin modalități diferite (Cookies, JWT, Local storage, Caching)